

RENANCE

---

# FEDERAL UNIVERSITY OF TECHNOLOGY, AKURE

COS 201 — Past Questions & Materials

**by Renance**

---

Course: COS 201

Institution: Federal University of Technology, Akure

Sources in this pack: 6

Includes past questions and lecture notes

---

This study pack was compiled and restructured by Renance for COS 201 students at Federal University of Technology, Akure. It brings together 6 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: [renance.app](https://renance.app)

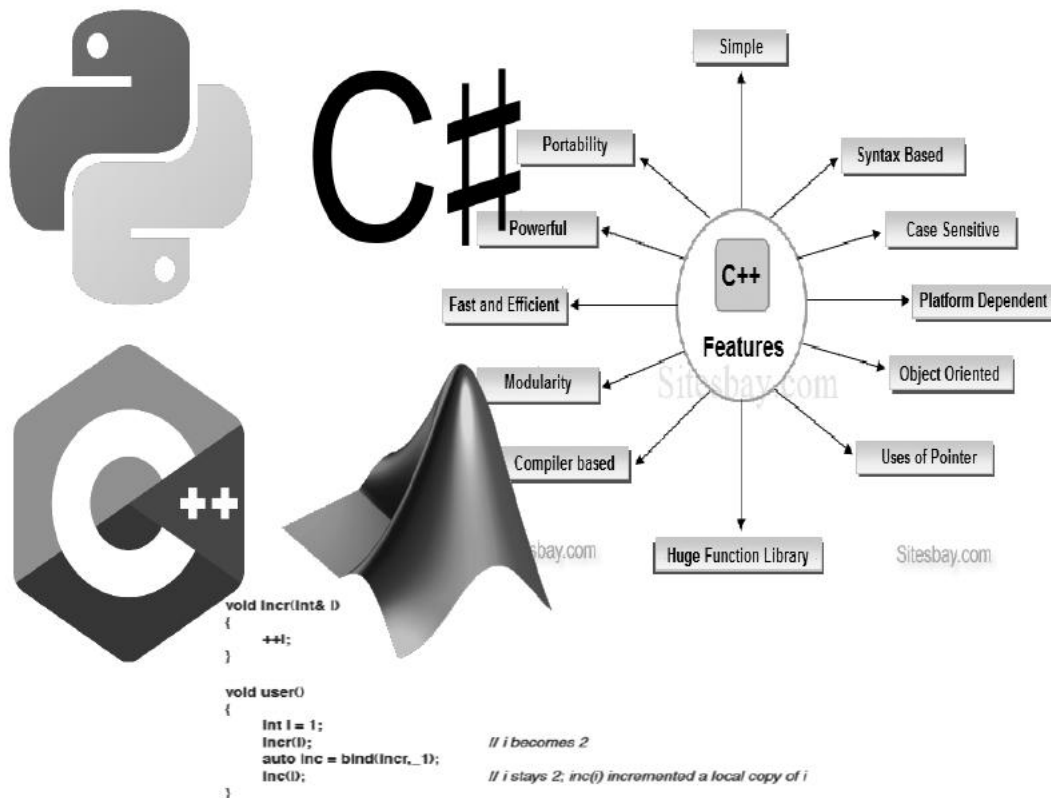
Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

# Paper 1 of 6 — Cyssa hub200 lvi materialsCOS 201Document from Tdexy.pdf

Lecture/Notes · Google Drive harvest

## COMPUTER PROGRAMMING I (C++)



**AJAYI Olusola Olajide, PhD**

OND, B.SC., M.SC., PhD  
MACM, MIEEE, MPSF, MCSTA, MIACSIT, MMCS  
(Lecturer, CSC Dept., AAUA)

[olusola.ajayi@aaua.edu.ng](mailto:olusola.ajayi@aaua.edu.ng)

[ajayicomputer@gmail.com](mailto:ajayicomputer@gmail.com)

[ajayiprogrammer@gmail.com](mailto:ajayiprogrammer@gmail.com)

## CHAPTER ONE

### INTRODUCTION TO C++ PROGRAMMING

#### 1.1 Introduction

C++ is an object-oriented programming language, which was built upon the success and the observed limitations of C programming language.

#### 1.2 The Structure

In accordance with the principle that guides good and programming language, C++ has its structure thus:

- The Header
- The Declaration Section
- The Body / Program Statement

##### The Header

```
#include<iostream>

using namespace std;

Main()

{

...code goes here

}
```

##### The Declaration Section

The content of this section is dependent upon DATA TYPES.

DATA TYPES refer to the nature of data that is stored unto a variable. In C++, we have five (5) major types:

- i. Integer: containing numeric data
- ii. Float: containing real value
- iii. Double: containing real value in greater magnitude
- iv. String: acceptable sequence of character at a time
- v. Char: accepting a character at a time

**I/O Statement****Output Statement**

The output statement in C++ is cout

e.g.

```
cout<<"The man is here"    //in the case of literal
```

```
cout<<sum;                //in the case of variable
```

**Input Statement**

The input statement in C++ is cin

e.g.

```
cin>>m;
```

```
cin>>m,n;
```

**Program Illustration I**

Write a C++ program that accepts two integer variables x and y, computes their sum and average.

**Solution**

```
#include<iostream>
using namespace std;
Main()
{
    int x,y, sum;
    float avg;
    cout<<"Enter 1st variable:";
    cin>>x;
    cout<<"Enter 2nd variable:";
    cin>>y;
    sum = x+y;
    avg = sum/2;
    cout<<"The sum is:"<<sum;
    cout<<"The average is:"<<avg;
}
```

**Lab Work I (All Lab Works are to be printed out on paper with computer)**

1. Write a C++ program that accepts three integer variables x, y, z, computes their sum and average.
2. Modify the program in (1) above to compute the product of the three variables.
3. Write an interactive C++ program that captures value for an integer variable X, self-multiply the variable to give its product.
4. Write an interactive C++ program that accepts value for an integer variable Y and computes its cube.
5. Write a C++ program that outputs: "AAU AKUNGBA-AKOKO" and spread same across the whole of your page/screen.

## CHAPTER TWO

### CONTROL STATEMENTS

- Control Statements (C.S.) are instructions that caused diversion in the normal flow of program execution. That is, they cause a kind of branching away from the normal pattern of code execution.
- Control Statements (C.S.) can also be programmatically defined as statements or codes that truncate the normal flow of program execution.

#### Types

- SELECTION C.S. (BRANCHING)
- ITERATION C.S. (LOOPING)

#### SELECTION C.S.

- A type of Control Statement that causes branching in the flow of program execution. In this control is transferred depending on the outcome of the conditional statement attached/specified. Simply calls it a DECISION Control Statement. It decides/judges based on the parameters supplied/fed in as conditions.

#### Types

##### (i) IF Statement

If Statement is a mono-directional selection control statement that presents only one option which can either be executed or no execution takes place.

If (conditional statement) executable statement;

e.g.

If (score>39) cout<<"Congrats, u passed!";

If (score<40) cout<<"Sorry, u failed, try again next session";

**(ii) IF-ELSE Statement**

If-else is a bi-directional selection control statement that presents two options in which only one is executed.

```
    If (conditional statement) executable statement1;  
    else executable statement2;
```

e.g.

```
If (score>39) cout<<"Congrats, u passed!";  
else cout<<"Sorry, u failed, try again next session";
```

**Program Illustration II**

Write a C++ program that accepts value for an integer m, detects and decides whether the variable is a positive number or a negative number.

**Solution**

```
#include<iostream>  
using namespace std;  
Main()  
{  
int m;  
cin>>m;  
if(m>0) cout<<"M is a positive integer";  
else cout<<"M is a negative number";  
}
```

**Lab Work II**

Write a C++ program that accepts value for an integer m, detects and decides whether the variable is a zero, a positive number or a negative number.

**(iii) IF-ELSEIF Statement**

If-else is a multi-directional selection control statement that presents many options in which only one is executed.

```

    If (conditional statement) executable statement1;
    elseif(conditional statement) executable statement2;
    else executable statement2;

```

e.g.

```

If (score>39) cout<<"Congrats, u passed!";
else if (score==0) cout<<"Script not found";
else cout<<"Sorry, u failed, try again next session";

```

**Switch-Case**

- Switch-Case Statement (A multi-options-one executed statement)
  - Switch(inputted/focus variable){

Case 1: executable statement I;

```
break;
```

Case 2: executable statement II;

```
break;
```

```

.
.

```

default: ....any instructional code goes here}

**Program Illustration III**

Write a C++ program that captures values for two integer variables num1 and num2, presents operation options that can be performed on them.

**Solution**

```
#include <iostream>
using namespace std;

main() {
    int choice_operation;
    int num1, num2;
    int sum, subtr, multi;
    float divi;

    //input/capture values for the variables first
    cout<<"Enter value for the first variable: "; cin>>num1;
    cout<<"Enter value for the second variable: "; cin>>num2;

    //present the operation options now...
    cout<<"Input 1 - Summation, 2 - Subtraction, 3 - Division, 4 - Multiplication";
    cin>>choice_operation;

    switch(choice_operation){
        case 1:
            cout<<"U choosed addition operation"<<endl;
            sum = num1+num2;
            cout<<"The summation of the two variables is= "<<sum;
            break;//to avoid fall thru
        case 2:
            cout<<"U decided on subtraction operation"<<endl;
            subtr = num1 - num2;
            cout<<"The difference between the two variables is= "<<subtr;
            break; //to avoid fall thru
        case 3:
            cout<<"U selected division operation"<<endl;
            divi = num1/num2;
            cout<<"The division resulted into= "<<divi;
            break; //to avoid fall thru
        case 4:
            cout<<"U opted for multplication operation"<<endl;
            multi = num1*num2;
            cout<<"The product of the two variables is= "<<multi;
            break; //to avoid fall thru
        default: cout<<"Sorry, redo, U selected a wrong/invalid no"; //to handle exception to the cases
    }
    return 0;
}
```

**Lab Work III**

1. Get to your bank, understudy the typical components of an ATM and how they work (e.g. withdrawal, balance, recharge, deposit, pay bill etc) and using C++ develop such an ATM using Switch-Case that will prevent/stop you from henceforth going to bank to transact, but to use your own built app.
2. Do you have a DSTV at home/office (you should), else browse to study how DSTV Subscription payment options are accessed and transactions done. After your survey and study, come up with a C++ designed DSTV Subscription System using Switch-Case Statement.

**ITERATION C.S.**

These are control statements that are used for repetitive execution of program statements. They are otherwise called Repetitive Control Statement.

We have:

- i. Do-While Statement
- ii. While Statement
- iii. FOR Statement

***Do-while***

These are post-test iterative statements in which the program statements are atleast executed once before the attached/stated condition is tested. This implies this statement does not take cognisance of the condition specified before executing the corresponding program statement.

**Program Illustration IV**

Write a simple but wise C++ program that demonstrates the use and lapses of DO statement.

**Solution**

```
#include<iostream>
using namespace std;

int main(int argc, char** argv) {
    int y=0;

    cout<<"For now, my current value is "<<y<<endl;
    do{
        cout<<endl<<"But DO Statement says: I am +1 now"<<endl;
    }while (y>1);
    cout<<endl<<"Compare the two statements, can u see the inefficiency of DO Statement? O ma n be ju!";
    return 0;
}
```

**While Statement**

This is the direct opposite of DO Statement. It is a pretest iterative control statement. Implying, this statement first test the attached/specified/given conditional statement before executing the program statement.

**Program Illustration V**

Write a C++ program that computes the sum of the first five (5) even integers.

**Solution**

```
#include <iostream>
using namespace std;

/* run this program using the console pauser or add your own getch, system("pause") or input loop */

int main(int argc, char** argv) {
    int evennum=2; // the beginning point of even nos
    int sumeven=0; //accumulator for the even summation
    int n=1; //the counter variable bin initialized to 1

    while (n<=5){
        sumeven = sumeven + evennum;
        evennum= evennum + 2; //increment evennum by 2. This can also be written as:
                                evennum+=2;
        n = n + 1; //This can also be written as n++;
    }
    cout<<"The sum of the first five even integers is="<<sumeven;
    return 0;
}
```

**Lab Work IV & V**

1. Convert the program in Program Illustration V using DO Statement.
2. Modify the program in Program Illustration V to accommodate for the first ten (10) odd integers using WHILE Statement.

**FOR Statement**

This is an iterative control statement that is used for executing program statement repeatedly in a particular number of times specified.

It has three (3) parameters:

*i. The initial value, ii. The terminating/conditional value, iii. The incremental value*

*e.g.*

```
for(int i=1; i<=10; i++)
```

For instance,

**Program Illustration VI**

Code to repeatedly print out the literal and for Jericho wall to fall (7 times):

***"Hallelujah for the Lord God the Omnipotent reigneth"***

U will code as follows:

```
#include<iostream>
using namespace std;

main()
{
    for(int i=1; i<=7; i++)
    {
        cout<<"Hallelujah for the Lord God the Omnipotent reigneth";
        cout<<endl; //this brings the cursor to the next line b4 d next print?
    }
    return 0;
}
```

**Lab Work VI**

1. Modify the program in Program Illustration V to accommodate for the first ten (10) odd integers but using FOR Statement.
2. Again, modify the program in Program Illustration V, this time to accommodate for the first ten (20) even integers using FOR Statement.
3. Write a C++ program that implements a score-grade application using if-elseif statement (use one course/one score).
4. With your knowledge of iterative statement now, modify the program in (3) above to cater for ten (10) students (use three courses/three scores).

## CHAPTER THREE

### ARRAY

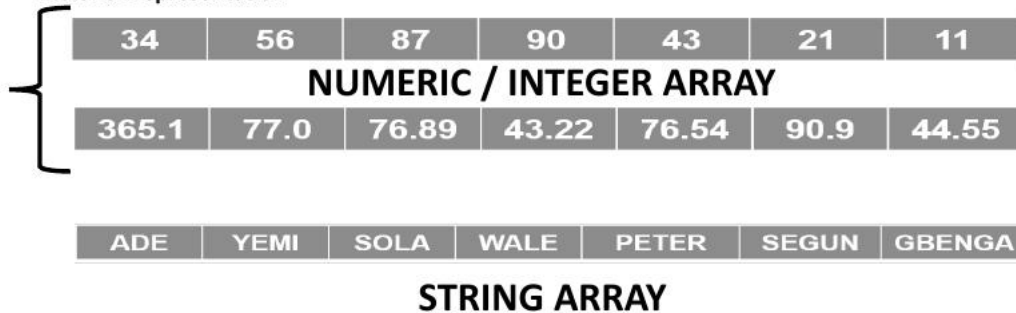
In this chapter, we shall be considering the following subheadings:

- i. DEFINITION
- ii. DIVISIONS
- iii. DECLARATIONS
- iv. DEMONSTRATIONS

#### 3.1 DEFINITION

- An array is a collection of data element, captured with the same name (storage location name) and having the same data type.
- The nature of data stored in an array depicts the type of array been created (i.e. numeric array, string array etc.)

##### Pictorial Representation



#### 3.2 DIVISION

We have two divisions of array:

- One Dimensional Array OR Linear Array
- Multidimensional Array OR Matrix

**Linear Array**

- An array that consist of a block of data, arrange in a row-dimensional form/horizontal form.

|    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|
| 34 | 56 | 87 | 90 | 43 | 21 | 11 |
|----|----|----|----|----|----|----|

**Format**

- To declare a linear array, we have
  - data type arrayname [size of the array];

e.g.

```
intmyscores[10];
```

To capture data into array, note that, ARRAY INDEX starts from 0 and not 1. for instance,

**Program Illustration VII**

To input the above array specification, you will code as:

```
#include<iostream>
using namespace std;
Main()
{
    intmyscores[10];
    for(inti=0; i<10; i++)
    {
        cout<<"Enter Score "<<i+1<<":"; cin>>myscores[i];
    }
    return 0;
}
```

**Program Illustration VIII***...solving similar problem but with n number of elements*

```

#include<iostream>
using namespace std;
Main()
{
    int n;
    cin>>n;
    intmyscores[n];
    for(inti=0; i<n; i++)
    {
        cout<<"Enter Score "<<i+1<<"."; cin>>myscores[i];
    }
    return 0;
}

```

**Arithmetic and Logical Operations on Array**

Some of the arithmetic operations that can be performed on ARRAY include but not limited to:

- Summation
- Multiplication
- Comparison

Such problems include:

- Largest element in an array dataset
- Summation or aggregate of an array data set
- Sorting of array dataset

**Program Illustrations IX-XI**

- Arithmetic Operations on two or more linear arrays (e.g. summation etc.)
- Real Life Application – Selection of Applicants for Vacant Positions
- Fabonacii is a number series in which the 3<sup>rd</sup> number is generated from the addition of the two previous numbers which are usually given as 0 and 1. How can u solve this for series of numbers (e.g. 20 series)?

**Program Illustrations IX**

- Arithmetic Operations on two or more linear arrays (e.g. summation etc.)

**Solutions****Summation of two numeric linear arrays with 5 elements**

```
#include <iostream>

using namespace std;

int array1[5],array2[5];
int newarray[5];

//input their elements
for (inti=0; i<5; i++)
{
    cout<<"Enter value for element "<<i+1<<" of Array 1: ";cin>>array1[i];
    cout<<"Enter value for element "<<i+1<<" of Array 2: ";cin>>array2[i];
    newarray[i]=array1[i]+array2[i];
}

cout<<"The elements of ARRAY1 are: "<<endl;
for(inti=0;i<5;i++){
    cout<<array1[i]<<" , ";
}

cout<<"\n"<<"The elements of ARRAY2 are: "<<endl;
for(inti=0;i<5;i++){
    cout<<array2[i]<<" , ";
}

cout<<"\n"<<"The SUMMATION of the above two array elements are: "<<endl;
for(inti=0;i<5;i++){
    cout<<newarray[i]<<" , ";
}

return 0; }
```

**Program Illustration X****Real Life Application – Selection of Applicants for Vacant Positions**

```

#include <iostream>

using namespace std;

int n;

//specify no of applicants
cout<<"Specify number of applicants: ";cin>>n;

intappid[n],appage[n];
char appgrade[n];
for(inti=0;i<n;i++)
{
    cout<<"Enter applicant "<<i+1<<" ID: ";cin>>appid[i];
    cout<<"Enter applicant "<<i+1<<" AGE: ";cin>>appage[i];
    cout<<"Enter applicant "<<i+1<<" GRADE: (D-Distinction, U-Upper, L-Lower, P-Pass,T-3rd
Class ";cin>>appgrade[i];
}
cout<<"\n"<<"SELECTED APPLICANTS ARE: "<<endl;
for(inti=0;i<n;i++)
{
    if((appage[i]<=26)&& (appgrade[i]=='D' | appgrade[i]=='U'))
        cout<<"Candidate "<<appid[i]<<endl;
    else continue;
}
return 0;
}

```

**Program Illustration XI****Fabonacii**

```

#include <iostream>
using namespace std;
Main(){
    int fabo[20];
    int fabo[0] = 0;
    int fabo[1] = 1;
    for(int i=2;i<20;i++)
    {
        fabo[i] = fabo[i-2]+fabo[i-1];
    }
    //outputting the fabo series
    for(int i=0; i<20;i++)
    {
        cout<<"\tfabo[i];
    }
    return 0;
}

```

**Lab Work VII - XI**

- Given the underlisted initialized original array elements (values), code in C++ to output the values in your output. NB: (1) The array size was not given. (2) The elements are to be inputted as specified. (3) The output is to be in the reverse order of the inputs

myvalues = {8, 5, 3, 5, 7, 2, 3, 4, 5, 6, 1, 17, 11, 19}

- Given

mybox = {4, 3, 8, 5, 7, 2}, sort and output this array elements in DESCENDING ORDER

(i.e. from highest to lowest

**Matrix**

Matrix is declared thus:

`int mymatrix[3][4]`, meaning a matrix containing 3 rows of elements and 4 columns of elements

e.g.

|   |   |   |   |
|---|---|---|---|
| 6 | 7 | 9 | 5 |
| 5 | 4 | 3 | 9 |
| 1 | 4 | 5 | 8 |

**Initialization**

If initialized, the format is:

`mymatrix = {2,5,6,8,4},{9,0,3,2,1},{8,9,3,4,7}`

Which implies a matrix of 3 rows and 5 columns

e.g.

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 5 | 6 | 8 | 4 |
| 9 | 0 | 3 | 2 | 1 |
| 8 | 9 | 3 | 4 | 7 |

**Programming Illustration XII****Inputting and Outputting Matrix (e.g. Row 3, Column 4)**

```
#include <iostream>
using namespace std;
int main(int argc, char** argv) {
    int mymatrix[3][4];
    //inputting....
    for(int i=0;i<3;i++)
    {
        for(int j=0;j<4;j++)
        {
            cout<<"Input Element in Row "<<i+1<<" Column "<<j+1<<" : "; cin>>mymatrix[i][j];
        }
    }
    //outputting...

    for(int i=0;i<3;i++)
    {
        cout<<endl;
        for(int j=0;j<4;j++)
        {
            cout<<mymatrix[i][j]<<" ";
        }
    }
    return 0;
}
```

**Programming Illustration XIII****Transposing a Matrix**

```

#include <iostream>
using namespace std;
/* run this program using the console pauser or add your own getch, system("pause") or input loop */
int main(int argc, char** argv) {
    int m,n;
    cout<<"Specify the matrix row dimension :"; cin>>m;
    cout<<"Specify the matrix column dimension :"; cin>>n;

    int ourmatrix[m][n];

    //inputting....
    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            cout<<"Input Element in Row "<<i+1<<" Column "<<j+1<<" : "; cin>>ourmatrix[i][j];
        }
    }
    //outputting...

    cout<<"\n"<<"This is the original matrix: "<<endl;
    for(int i=0;i<m;i++)
    {
        cout<<endl;
        for(int j=0;j<n;j++)
        {
            cout<<ourmatrix[i][j]<<" ";
        }
    }

    cout<<"\n"<<"And lo, the transpose of the original matrix: "<<endl;
    for(int i=0;i<m;i++)
    {
        cout<<endl;
        for(int j=0;j<n;j++)
        {
            cout<<ourmatrix[j][i]<<" ";
        }
    }
    return 0;
}

```

20

**Program Illustration XIV****Arithmetic Operations on Matrix**

```

#include <iostream>

using namespace std;

/* run this program using the console pauser or add your own getch, system("pause") or input loop */

int main(int argc, char** argv) {
    int m, n;
    cout << "Specify the matrix row dimension: "; cin >> m;
    cout << "Specify the matrix column dimension "; cin >> n;

    int hismatrix[m][n], hermatrix[m][n], theirmatrix[m][n];

    //inputting....
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << "Input Element in Row "<< i + 1 << " Column "<< j + 1 << " for Hismatrix: "; cin >> hismatrix[i][j];
            cout << "Input Element in Row "<< i + 1 << " Column "<< j + 1 << " for Hermatrix: "; cin >> hermatrix[i][j];
            theirmatrix[i][j] = hismatrix[i][j] + hermatrix[i][j];
        }
    }

    //outputting....
    cout << "\n" << "HISMATRIX: " << endl;
    for (int i = 0; i < m; i++)
    {
        cout << endl;
        for (int j = 0; j < n; j++)

```

```
        {
            cout<<hismatrix[i][j]<<" ";
        }
    }
    cout<<"\n"<<"HERMATRIX: "<<endl;
    for(int i=0;i<m;i++)
    {
        cout<<endl;
        for(int j=0;j<n;j++)
        {
            cout<<hermatrix[i][j]<<" ";
        }
    }
    cout<<"\n"<<"MARRYING THEM TOGETHER, THEIRMATRIX: "<<endl;
    for(int i=0;i<m;i++)
    {
        cout<<endl;
        for(int j=0;j<n;j++)
        {
            cout<<theirmatrix[i][j]<<" ";
        }
    }

    return 0;
}
```

**Lab Work XII - XIV**

1. Modify the matrix arithmetic operation by:
  - i. Code the inputting of hismatrix and hermatrix elements separately
  - ii. Performs subtraction operation on the two matrices to give theirdiffmatrix
  - iii. Perform division operation on the two matrices to give theirdivmatrix
  - iv. Compute and output the inverse of each matrix

## 2. Given

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 0 \end{bmatrix}$$

and

$$B = [2, 3]$$

Compute and output:

- i.  $A * B$
- ii.  $A/B$
- iii.  $A^2$

## 3. Capture the below given value for Matrix A:

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 0 \end{bmatrix}$$

Then, compute and output:

## i. row addition of Matrix A's elements

$$\begin{aligned} \text{e.g.} \quad & 9 \\ & 18 \\ & 17 \end{aligned}$$

## ii. Column addition of Matrix A's elements

$$\begin{aligned} \text{e.g.} \quad & 15 \quad 18 \quad 11 \end{aligned}$$

## CHAPTER FOUR

### FUNCTIONS AND LIBRARIES

In this chapter, we shall be considering the following subheadings:

- i. DEFINITION
- ii. DECLARATIONS
- iii. DIVISIONS
- iv. DEMONSTRATIONS

#### 4.1 DEFINITION

- A Function can be defined as a self-contained program that is written to perform a specific and well-defined task. A self-contained program implies it's a program that can be written separately on its own and can still run. However, because a self-contained program sometimes needs the service(s) of the larger arena, it is mostly run alongside the main program. It serves/runs as a subordinate (sub) program to the main program.
- In some programming languages, it is called subroutine (e.g. Qbasic, Fortran etc.), Procedures in some (e.g. Pascal, Cobol etc.), and Classes in others (e.g. C++, Java, C# etc.)
- NB: Classes can contain lots of Functions. They are like higher functions
- The Function is usually regarded as the CALLED Program, while the main program that calls it is called the CALLING Program.

Now, how is this relationship defined?

## 4.2 DECLARATION

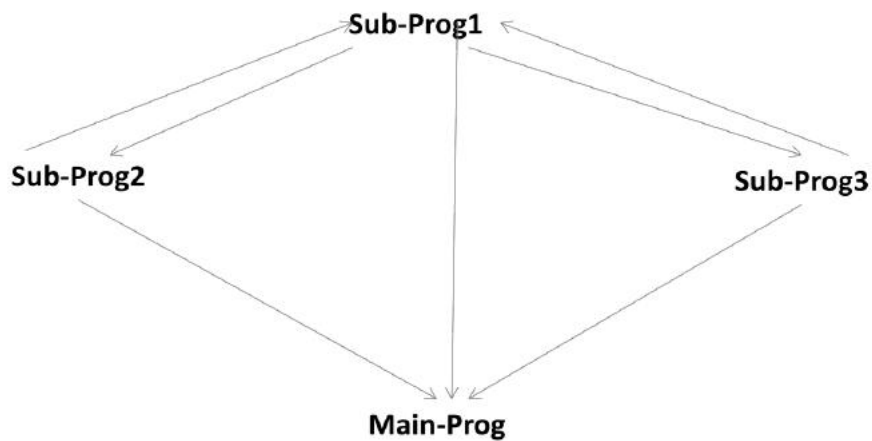
There are two ways by which a function can be declared:

- i. Using Bottom-Up Approach
- ii. Using Top-Down Approach

### BOTTOM-UP APPROACH

- This approach implies a coding style/software development approach in which subprogram/modules are written before the main program. So, in this approach, the function(s) comes before the main program. This is the normal / customary approach of writing in functions.

#### Pictorial Representation



#### Program Illustration XV

function data type ...function name (function parameters/variable & their data types)

```
{  
The code goes here...  
}
```

main function

```
{
```

The code goes here...

```
}
```

Let's see example now.....

```
int thelargest(int x,int y)
```

```
{
```

```
    if (x>y)return x;
```

```
    else return y;
```

```
}
```

#### **Program Illustration XVI**

```
#include <iostream>
```

```
using namespace std;
```

```
int main(int argc, char** argv) {
```

```
    int num1,num2;
```

```
    //input
```

```
    cout<<"Enter value for first no: ";    cin>>num1;
```

```
    cout<<"Enter value for second no: "; cin>>num2;
```

```
    //find the largest now.....
```

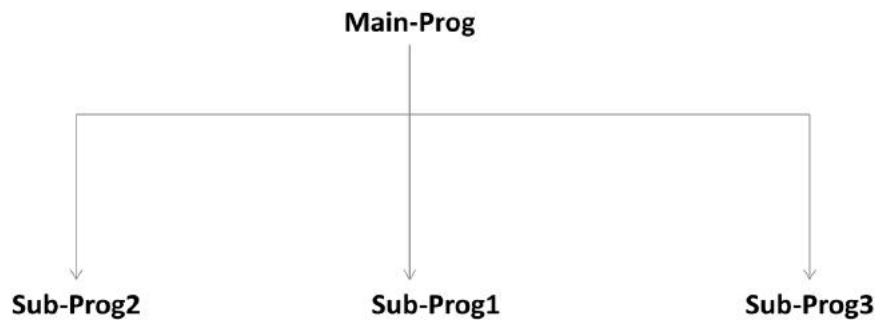
```
    cout<<"The largest of the two values inputted is:" <<thelargest(num1,num2);
```

```
    return 0;
```

```
}
```

**TOP-DOWN APPROACH**

- This approach implies having the main program coded first, then the subprograms. So, we mean here, that the main function will be written first, then below it we have the sub-functions coded. Note that it has been established that the bottom-up is the ideal way of coding in functions. However, to use this method, it implies we have to adopt what we call FUNCTION PROTOTYPE.
- FUNCTION PROTOTYPE means indicating/specifying the function name above the code level of the main function.
- NB: If this is not done, the compiler will find it difficult to locate your subprogram/function

**Pictorial Representation****Program Illustration XVII**

```
function prototype definition;  
main function  
{  
  The code goes here...  
}  
Function / Subprog  
{  
  The code goes here...  
}  
Let's see example now.....
```

```

#include <iostream>
using namespace std;
int main(int argc, char** argv) {
    int num1,num2;
    //input
    cout<<"Enter value for first no: ";    cin>>num1;
    cout<<"Enter value for second no: "; cin>>num2;
    //find the largest now.....
    cout<<"The largest of the two values inputted is:" <<thelargest(num1,num2);

    return 0;
}
int thelargest(int x,int y)
{
    if (x>y)return x;
    else return y;
} //U will run into problem here

```

- But let's have it this way....complete definition

#### Program Illustration XVIII

```

int thelargest(int, int);
#include <iostream>
using namespace std;
int main(int argc, char** argv) {
    int num1,num2;
    //input
    cout<<"Enter value for first no: ";    cin>>num1;
    cout<<"Enter value for second no: "; cin>>num2;
    //find the largest now.....
    cout<<"The largest of the two values inputted is:" <<thelargest(num1,num2);
    return 0;
}
int thelargest(int x,int y)
{
    if (x>y)return x;
    else return y;
}

```

### 4.3 DIVISION

Function can be viewed to be in two forms/divisions:

- i. Programmer-Defined Function (PDF) OR User-Defined Function (UDF)
- ii. System-Defined Function (SDF) OR Developer/Designer-Defined Function (DDF)

PDFs are functions that are written by programmers/coders, while SDFs are functions written by computer manufacturers or compiler designers and stored as libraries.

What we have illustrated thus far is PDF. We'll still go through some few more codes on this before SDFs.

#### Program Illustration XIX

##### FACTORIAL Program

```
long myfactorial(int x){  
    int fact=1;  
    for(int i=1;i<=x;i++){  
        fact = fact*i;  
    }  
    return fact;  
}  
  
#include <iostream>  
  
using namespace std;  
  
int main(int argc, char** argv) {  
    int n;
```

```

cout<<"Enter d factorial number: ";cin>>n;

cout<<"The factorial of "<<n<<"is "<<myfactorial(n);

return 0;

}

```

### Program Illustration XX

#### PERMUTATION Program

```

long myfactorial(int x){

    int fact=1;

    for(int i=1;i<=x;i++){

        fact = fact*i;

    }

    return fact;

}

#include <iostream>

using namespace std;

int main(int argc, char** argv) {

    int n,m;

    cout<<"Enter d factorial number: ";cin>>n;

    cout<<"Enter d 2nd factorial number: ";cin>>m;

    cout<<"The permutation of "<<n<<" and "<<m<<" is "<<myfactorial(n)/(myfactorial(n-m));

    return 0;
}

```

30

**Lab Work XV - XX**

- Extend the illustrated factorial/permutation programs by computing the combination of  $n$  and  $m$ , where the value of  $n$  is greater than  $m$ .
- Write a C++ function program that inputs some past years (e.g. 1990-2015), detects and outputs the years that are LEAP YEAR.

**SDFs now....**

- These vary according to their libraries. For instance, the followings are SDFs under the mathematical library called `math.h`:
  - ✓ Sin (`sin`)
  - ✓ Tangent (`tan`)
  - ✓ Cosine (`cos`)
  - ✓ Square root (`sqrt`)
  - ✓ Power (`pow`)
- Now, for most computations using SDFs, you may not need have separate functions, since they are functions on their own. What we mean by this is that, you can just have them coded within a function and the job will still be done for you. OK? Let's justify this now with a simple program.

**Program Illustration XXI**

```
#include <iostream>

#include<math.h> //attempt omitting this 1st, then after running, recode/re-insert
using namespace std;

    int x, y;
    cout<<"Enter the base number: "; cin>>x;
    cout<<"Enter the raise number: "; cin>>y;
    cout<<"The power of "<<x<<" to "<<y<<" is: "<<pow(x,y);
    return 0;
}
```

**HOW???**

- .....rand()%a value
- e.g. : .....rand()%6 //in case of a dice
- rand()%6 however implies : generate digits btw 1 to 6-1 i.e. btw 1-5.
  - So, if u want 6 included, u will have it as: rand()%6+1 OR 1+ rand()%6

Let's try out some codes for illustrations

**Program Illustration XXII**

```
#include<iostream>
#include<stdlib.h>
using namespace std;
{
int main(int argc, char** argv)
    cout<<"toss is: "<<1+rand()%6<<endl;
    cout<<"END TOSSING";
return 0;
}

//run severally or put within loop & observe d returned value(s)
```

**Attention!**

- Applying rand effectively & appropriately involves the use of a sister function command that acts as check value for rand. It is called SRAND. Note that to use this command effectively also entails assigning it to numerical variable or value like time; which will implies using TIME library as well.

**Program Illustration XXIII – Tossing of a dice**

```
#include <iostream>

#include<stdlib.h>

#include<time.h>

using namespace std;

main() {

int control = time(NULL);

srand(control);

    cout<<"toss is: "<<1+rand()%6;

    cout<<"END TOSSING";

return 0;

}
```

**Program Illustration XXIV – Tossing of 2 dice**

```
#include <iostream>

#include<stdlib.h>

#include<time.h>

using namespace std;

main() {

int control = time(NULL);

srand(control);

    cout<<"tossed is: "<<1+rand()%6;cout<<" and "<<1+rand()%6<<endl;

    cout<<"END TOSSING";

return 0;

}
```

**Program Illustration XXV****Double LUDO Players – 1 dice – 10x Throw**

```
#include <iostream>

#include<stdlib.h>

#include<time.h>

using namespace std;

main() {

int control = time(NULL);

srand(control);

for (int i=1;i<=10;i++){

cout<<"toss1 "<<" throw "<<i<<" is: "<<1+rand()%6<<"\t\t"<<"toss2 "<<" throw "<<i<<" is:
"<<1+rand()%6<<endl;

}

cout<<endl<<"END TOSSING";

return 0;
```

**Lab Work XXI - XXV**

- In the olden Indian tradition, marriage is conducted by way of throwing dicing object. If the object thrown sums up to 7, it is a perfect marriage, if less than, unmarriageable, if greater, uncommon match, if equals to 0, not yet fit for marriage. Code to computerize this.
- Among many libraries available in C++ is the time.h which contains functions like ctime(), clock() etc; ctype.h containing isspace(), isnumeric(), isdigit(), toupper(), tolower(); math.h containing sin(), cos(), tan() etc. Design a program on your own that will demonstrate (in one program) the use of all these functions. Go ahead, that is what it takes to be a programmer.

## CHAPTER FIVE

### STRING

- A String is a sequence of characters stored in memory.
- They are stored in the library `<string.h>`
- They are declared as *char* and inputted as array of character either as initialized or inputted set of data.

#### 5.1 STRING FUNCTIONS

- There are lots of functions you can have accessed to through the STRING Functions. Some of these are:
  - `strlen()`
  - `strcat()`
  - `strncat()`
  - `strcpy()`
  - `strncpy()`
  - `strtok()`
- **STRLEN:** Is used to capture/return the length of string. It returns numeric value.
- **STRCAT:** Is used to concatenate/merge two or more string/character variables together. It returns string/character value.
- **STRNCAT:** ditto **STRCAT** but allows the programmer to specify the number (N) of characters to be merge.
- **STRCPY:** Is used to copy/insert/append a string into another one thereby overwriting the other. It returns string or character value.

- STRNCPY: ditto STRCPY but in addition it allows number of string to be copied/inserted/append(N) to be specified.
- STRTOK: Is used to filter each token in a character string. That is used it is used to filter each word (before space).

## 5.2 PROGRAM DEMONSTRATIONS

### Program Illustration XXVI

```
#include <iostream>

#include <string.h>

using namespace std;

int main(int argc, char** argv) {

    char myname[]="This is my story, this my song";

    char myname2[]="Praising my Saviour for this course";

    char* t;

    cout<<"The initialized string is: "<<endl;

    cout<<myname;

    cout<<"\n"<<"The length of the string is: "<<strlen(myname);

    cout<<"The 2nd initialized string is: "<<endl;

        cout<<myname2;

        cout<<"\n"<<"The length of the string is: "<<strlen(myname2);

    }

    cout<<"\n"<<"The merging is: "<<endl;
```

```
        cout<<strcat(myname,myname2);
    }

    cout<<"The tokens in this statement are: "<<endl;

    t = strtok(myname, " ");

    while (t){

        cout<<"\t"<<t<<"\n";

        t = strtok(NULL, " ");

    }

    return 0;
}
```

### Class Exercise

Given the string variables:

Mine = 'AJAYI Olusola Olufunke'

You = 'Testimony Timothy Toluwani'

Code in C++ to output:

- (i) Length of Mine
- (ii) Length of You
- (iii) Concatenation of Mine and You
- (iv) Concatenate Mine and Timothy
- (v) Concatenate Olusola and Toluwani
- (vi) Concatenate AJAYI and Testimony

**Lab Work XXVI**

- (1) On your own, modify the program under the previous exercise (above) to suit yourself and your family.
- (2) Code in C++ to find:
  - a. The length of "All hails ADEKUNLE AJASIN VARSITY"
  - b. Merge your birth month with the current month
  - c. Find the number of xter in each month of the year
  - d. Compute the number of xter in the names of the state in Nigeria
  - e. Compute the number of xter in the names of the countries in Africa

## **Paper 2 of 6 — Cyssa hub200 lvi materialsCOS 201IMG-20260120-WA0168.jpg**

Lecture/Notes · Google Drive harvest

---

## **Paper 3 of 6 — Cyssa hub200 lvi materialsCOS 201IMG-20260127-WA0079.jpg**

Lecture/Notes · Google Drive harvest

---

## **Paper 4 of 6 — Cyssa hub200 lvi materialsCOS 201IMG-20260212-WA0014.jpg**

Lecture/Notes · Google Drive harvest

---

## Paper 5 of 6 — Cyssa hub200 lvl materialsCOS 201c manual (incomplete).pdf

Lecture/Notes · Google Drive harvest

### CONTENTS

#### Part One (C++):

**Module One: Introduction to C++ Programming**

**Module Two: Control Statements in C++**

**Module Three: Array**

**Module Four: Functions and Libraries**

**Module Five: String**

#### Part Two (JAVA):

**Module Six: Programming with Java**

**Module Seven: Control Statements in Java**

**Module Eight: Array Programming in Java**

**Module Nine: OOP Features**

#### Part Three (C#):

**Module Ten: The Structure of C#**

#### Part Four (Python):

**Module Eleven: The Structure of Python**

**Module Twelve: Control Structure in C# and Python**

**Module Thirteen: Built-in Functions in Python**

## MODULE ONE

## INTRODUCTION TO C++ PROGRAMMING

**Introduction**

C++ is an object-oriented programming language, which was built upon the successes and the observed limitations of C programming language.

**The Structure**

In accordance with the programming style principle that guides every good and standard programming language, C++ has its structure thus:

- The Header
- The Declaration Section
- The Body / Program Statement

**The Header**

```
#include<iostream>
using namespace std;
Main()
{
...code goes here
}
```

**The Declaration Section**

The content of this section is dependent upon DATA TYPES.

DATA TYPES refer to the nature of data that is stored into a variable. In C++, we have five (5) major types:

- i. Integer: containing numeric data
- ii. Float: containing real value
- iii. Double: containing real value in greater magnitude
- iv. String: acceptable sequence of character at a time
- v. Char: accepting a character at a time

**I/O Statement****Output Statement**

The output statement in C++ is cout

e.g.

```
cout<<"The man is here" //in the case of literal
cout<<sum; //in the case of variable
```

**Input Statement**

The input statement in C++ is cin

e.g.

```
cin>>m;
cin>>m,n;
```

**Program Illustration I**

Write a C++ program that accepts two integer variables x and y, computes their sum and average.

**Solution**

```
#include<iostream>
using namespace std;
Main()
{
int x,y, sum;
float avg;
cout<<"Enter 1st variable:";
cin>>x;
cout<<"Enter 2nd variable:";
```

```

cin>>y;
sum = x+y;
avg = sum/2;
cout<<"The sum is:"<<sum;
cout<<"The average is:"<<avg;}

```

**Lab Work I** (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)

1. Write a C++ program that accepts three integer variables x, y, z, computes their sum and average.
2. Modify the program in (1) above to compute the product of the three variables.
3. Write an interactive C++ program that captures value for an integer variable X, self-multiply the variable to give its product.
4. Write an interactive C++ program that accepts value for an integer variable Y and computes its cube.
5. Write a C++ program that outputs: "AAU AKUNGBA-AKOKO" and spread same across the whole of your page/screen.

*Coding Sheet (where this is not sufficient, please kindly print out and append)*

Program ID/No: \_\_\_\_\_ Start Time: \_\_\_\_\_ Run Time: \_\_\_\_\_

| Program Statement: |  |
|--------------------|--|
| 1.                 |  |
| 2.                 |  |
| 3.                 |  |
| 4.                 |  |
| 5.                 |  |
| 6.                 |  |
| 7.                 |  |
| 8.                 |  |
| 9.                 |  |
| 10.                |  |
| 11.                |  |
| 12.                |  |
| 13.                |  |
| 14.                |  |
| 15.                |  |
| 16.                |  |
| 17.                |  |
| 18.                |  |
| 19.                |  |
| 20.                |  |
| 21.                |  |
| 22.                |  |
| 23.                |  |
| 24.                |  |
| 25.                |  |

## MODULE TWO

## CONTROL STATEMENTS IN C++

- Control Statements (C.S.) are instructions that caused diversion in the normal flow of program execution. That is, they cause a kind of branching away from the normal pattern of code execution.
- Control Statements (C.S.) can also be programmatically defined as statements or codes that truncate the normal flow of program execution.

**Types**

- SELECTION C.S. (BRANCHING)
- ITERATION C.S. (LOOPING)

**SELECTION C.S.**

- A type of Control Statement that causes branching in the flow of program execution. In this control is transferred depending on the outcome of the conditional statement attached/specified. Simply calls it a DECISION Control Statement. It decides/judges based on the parameters supplied/fed in as conditions.

**Types****(i) IF Statement**

If Statement is a mono-directional selection control statement that presents only one option which can either be executed or no execution takes place.

If (conditional statement) executable statement;

e.g.

If (score > 39) cout << "Congrats, u passed!";

If (score < 40) cout << "Sorry, u failed, try again next session";

**(ii) IF-ELSE Statement**

If-else is a bi-directional selection control statement that presents two options in which only one is executed.

If (conditional statement) executable statement1;

else executable statement2;

e.g.

If (score > 39) cout << "Congrats, u passed!";

else cout << "Sorry, u failed, try again next session";

**Program Illustration II**

Write a C++ program that accepts value for an integer m, detects and decides whether the variable is a positive number or a negative number.

**Solution**

```
#include <iostream>
```

```
using namespace std;
```

```
Main()
```

```
{
```

```
int m;
```

```
cin >> m;
```

```
if (m > 0) cout << "M is a positive integer";
```

```
else cout << "M is a negative number";
```

```
}
```

**(iii) IF-ELSEIF Statement**

If-else is a multi-directional selection control statement that presents many options in which only one is executed.

If (conditional statement) executable statement1;

elseif (conditional statement) executable statement2;

else executable statement2;

e.g.  
 If(score>39) cout<<"Congrats, u passed!";  
 else if(score==0) cout<<"Script not found";  
 else cout<<"Sorry, u failed, try again next session";

### Switch-Case

- Switch-Case Statement (A multi-options-one executed statement)

```
Switch(inputted/focus variable){
```

```
Case 1: executable statement I;
```

```
    break;
```

```
Case 2: executable statement II;
```

```
    break;
```

```
default: .....any instructional code goes here}
```

### Program Illustration III

Write a C++ program that captures values for two integer variables num1 and num2, presents operation options that can be performed on them.

### Solution

```
#include<iostream>
using namespace std;
main() {
    int choice_operation;
    int num1, num2;
    int sum, subt, multi;
    float divi;
    //input/capture values for the variables first
    cout<<"Enter value for the first variable: "; cin>>num1;
    cout<<"Enter value for the second variable: "; cin>>num2;
    //present the operation options now...
    cout<<"Input 1 - Summation, 2 - Subtraction, 3 - Division, 4 - Multiplication";
    cin>>choice_operation;
    switch(choice_operation){
        case 1:
            cout<<"U chose addition operation"<<endl;
            sum = num1 + num2;
            cout<<"The summation of the two variables is="<<sum;
            break;//to avoid fall thru
        case 2:
            cout<<"U decided on subtraction operation"<<endl;
            subt = num1 - num2;
            cout<<"The difference between the two variables is="<<subt;
            break;//to avoid fall thru
        case 3:
            cout<<"U selected division operation"<<endl;
            divi = num1/num2;
            cout<<"The division resulted into="<<divi;
            break;//to avoid fall thru
        case 4:
            cout<<"U opted for multiplication operation"<<endl;
            multi = num1 * num2;
```

```

        cout<< "The product of the two variables is=" << multi;
        break; //to avoid fall thru
default: cout<< "Sorry, redo. U selected a wrong/invalid no"; //to handle exception to the cases
    }
    return 0;
}

```

### ITERATION C.S.

These are control statements that are used for repetitive execution of program statements. They are otherwise called Repetitive Control Statement.

We have:

- i. Do-While Statement
- ii. While Statement
- iii. FOR Statement

#### Do-while

These are post-test iterative statements in which the program statements are atleast executed once before the attached/stated condition is tested. This implies this statement does not take cognisance of the condition specified before executing the corresponding program statement.

#### Program Illustration IV

Write a simple but wise C++ program that demonstrates the use and lapses of DO statement.

#### Solution

```

#include<iostream>
using namespace std;
int main(int argc, char** argv) {
    int y=0;
    cout<<"For now, my current value is "<<y<<endl;
    do{
        cout<<endl<<"But DO Statement says: I am +1 now"<<endl;
    }while (y>1);
    cout<<endl<<"Compare the two statements, can u see the inefficiency of DO Statement? O man be ju!";
    return 0;
}

```

#### While Statement

This is the direct opposite of DO Statement. It is a pretest iterative control statement. Implying, this statement first test the attached/specified/given conditional statement before executing the program statement.

#### Program Illustration V

Write a C++ program that computes the sum of the first five (5) even integers.

#### Solution

```

#include <iostream>
using namespace std;
/* run this program using the console pauser or add your own getch, system("pause") or input loop */
int main(int argc, char** argv) {
    int evennum=2; // the beginning point of even nos
    int sumeven=0; //accumulator for the even summation
    int n=1; //the counter variable bin initialized to 1
    while (n<=5){
        sumeven = sumeven + evennum;
    }
}

```

```

        evennum=evennum+2; //increment evennum by 2. This can also be written as:
                                evennum+=2;
        n=n+1; //This can also be written as n++;
    }
    cout<<"The sum of the first five even integers is="<<sumeven;
    return 0;
}

```

### FOR Statement

This is an iterative control statement that is used for executing program statement repeatedly in a particular number of times specified.

It has three (3) parameters:

*i. The initial value, ii. The terminating/conditional value, iii. The incremental value*

e.g.

```
for(int i=1; i<=10; i++)
```

For instance,

### Program Illustration VI

Code to repeatedly print out the literal and for Jericho wall to fall (7 times):

*"Hallelujah for the Lord God the Omnipotent reigneth"*

U will code as follows:

```

#include<iostream>
using namespace std;

main()
{
    for(int i=1; i<=7; i++)
    {
        cout<<"Hallelujah for the Lord God the Omnipotent reigneth";
        cout<<endl; //this brings the cursor to the next line b4 d next print?
    }
    return 0;
}

```

### Lab Work II (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)

1. Get to your bank, understudy the typical components of an ATM and how they work (e.g. withdrawal, balance, recharge, deposit, pay bill etc) and using C++ develop such an ATM using Switch-Case that will prevent/stop you from henceforth going to bank to transact, but to use your own built app.
2. Do you have a DSTV at home/office (you should), else browse to study how DSTV Subscription payment options are accessed and transactions done. After your survey and study, come up with a C++ designed DSTV Subscription System using Switch-Case Statement.
3. Convert the program in Program Illustration V using DO Statement.
4. Modify the program in Program Illustration V to accommodate for the first ten (10) odd integers using WHILE Statement.
5. Modify the program in Program Illustration V to accommodate for the first ten (10) odd integers but using FOR Statement.
6. Again, modify the program in Program Illustration V, this time to accommodate for the first ten (20) even integers using FOR Statement.

7. Write a C++ program that implements a score-grade application using if-elseif statement (use one course/one score).
8. With your knowledge of iterative statement now, modify the program in (3) above to cater for ten (10) students (use three courses/three scores).

*Coding Sheet (where this is not sufficient, please kindly print out and append)*

Program ID/No: \_\_\_\_\_ Start Time: \_\_\_\_\_ Run Time: \_\_\_\_\_

| Program Statement: |  |
|--------------------|--|
| 1.                 |  |
| 2.                 |  |
| 3.                 |  |
| 4.                 |  |
| 5.                 |  |
| 6.                 |  |
| 7.                 |  |
| 8.                 |  |
| 9.                 |  |
| 10.                |  |
| 11.                |  |
| 12.                |  |
| 13.                |  |
| 14.                |  |
| 15.                |  |
| 16.                |  |
| 17.                |  |
| 18.                |  |
| 19.                |  |
| 20.                |  |
| 21.                |  |
| 22.                |  |
| 23.                |  |
| 24.                |  |
| 25.                |  |

## MODULE THREE

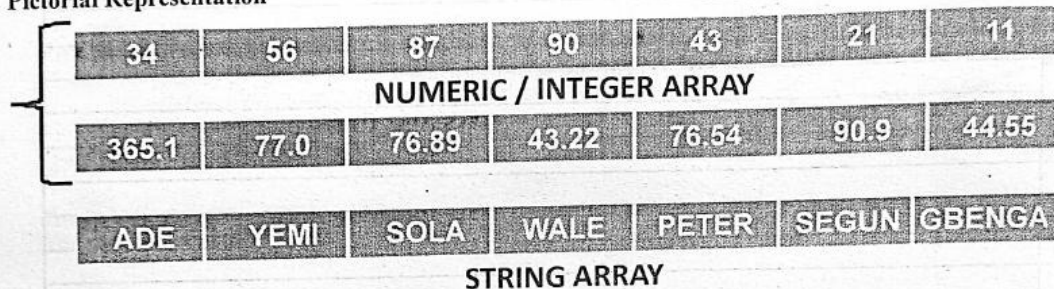
## ARRAY

In this chapter, we shall be considering the following subheadings:

- i. DEFINITION
- ii. DIVISIONS
- iii. DECLARATIONS
- iv. DEMONSTRATIONS

**DEFINITION**

- An array is a collection of data element, captured with the same name (storage location name) and having the same data type.
- The nature of data stored in an array depicts the type of array been created (i.e. numeric array, string array etc.)

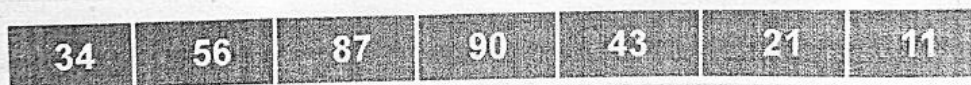
**Pictorial Representation****DIVISION**

We have two divisions of array:

- One Dimensional Array OR Linear Array
- Multidimensional Array OR Matrix

**Linear Array**

- An array that consist of a block of data. arrange in a row-dimensional form/horizontal form.

**Format**

- To declare a linear array, we have
  - data type arrayname [size of the array];

e.g.

```
intmyscores[10];
```

To capture data into array, note that, ARRAY INDEX starts from 0 and not 1. for instance,

**Program Illustration VII**

To input the above array specification, you will code as:

```
#include<iostream>
```

```
using namespace std;
```

```
Main()
```

```
{
    intmyscores[10];
    for(inti=0; i<10; i++)
    {
        cout<<"Enter Score "<<i+1<<" "; cin>>myscores[i];
    }
}
```

```
return 0;
}
```

### Program Illustration VIII

...solving similar problem but with n number of elements

```
#include<iostream>
using namespace std;
Main()
{
    int n;
    cin>>n;
    int myscores[n];
    for(int i=0; i<n; i++)
    {
        cout<<"Enter Score "<<i+1<<" "; cin>>myscores[i];
    }
    return 0;
}
```

### Arithmetic and Logical Operations on Array

Some of the arithmetic operations that can be performed on ARRAY include but not limited to:

- Summation
- Multiplication
- Comparison

Such problems include:

- Largest element in an array dataset
- Summation or aggregate of an array data set
- Sorting of array dataset

### Program Illustrations IX-XI

- Arithmetic Operations on two or more linear arrays (e.g. summation etc.)
- Real Life Application – Selection of Applicants for Vacant Positions
- Fabonacii is a number series in which the 3<sup>rd</sup> number is generated from the addition of the two previous numbers which are usually given as 0 and 1. How can u solve this for series of numbers (e.g. 20 series)?

### Program Illustrations IX

- Arithmetic Operations on two or more linear arrays (e.g. summation etc.)

### Solutions: Summation of two numeric linear arrays with 5 elements

```
#include <iostream>
using namespace std;
int array1[5], array2[5];
int newarray[5];
//input their elements
for (inti=0; i<5; i++)
{
    cout<<"Enter value for element "<<i+1<<" of Array 1: "; cin>>array1[i];
    cout<<"Enter value for element "<<i+1<<" of Array 2: "; cin>>array2[i];
    newarray[i]=array1[i]+array2[i];
}
cout<<"The elements of ARRAY1 are: "<<endl;
for(inti=0; i<5; i++){
    cout<<array1[i]<<" ";
}
```

```

cout<<"\n"<<"The elements of ARRAY2 are: "<<endl;
for(inti=0;i<5;i++){
    cout<<array2[i]<<" ";
}
cout<<"\n"<<"The SUMMATION of the above two array elements are: "<<endl;
for(inti=0;i<5;i++){
    cout<<newarray[i]<<" ";
}
return 0;}

```

### Program Illustration X

#### Solution: Real Life Application – Selection of Applicants for Vacant Positions

```

#include <iostream>
using namespace std;
int n;
//specify no of applicants
cout<<"Specify number of applicants: ";cin>>n;

intappid[n],appage[n];
char appgrade[n];
for(inti=0;i<n;i++)
{
    cout<<"Enter applicant "<<i+1<<" ID: ";cin>>appid[i];
    cout<<"Enter applicant "<<i+1<<" AGE: ";cin>>appage[i];
    cout<<"Enter applicant "<<i+1<<" GRADE: (D-Distinction, U-Upper, L-Lower, P-Pass, T-3rd Class
";cin>>appgrade[i];
}
cout<<"\n"<<"SELECTED APPLICANTS ARE: "<<endl;
for(inti=0;i<n;i++)
{
    if((appage[i]<=26)&& (appgrade[i]=='D'||appgrade[i]=='U'))
        cout<<"Candidate "<<appid[i]<<endl;
    else continue;
}
return 0;
}

```

### Program Illustration XI

#### Fabonacii

```

#include <iostream>
using namespace std;
Main(){
    int fabo[20];
    int fabo[0]=0;
    int fabo[1]=1;
    for(int i=2;i<20;i++)
    {
        fabo[i]=fabo[i-2]+fabo[i-1];
    }
    //outputting the fabo series
    for(int i=0;i<20;i++)
    {
        cout<<"\tfabo[i];
    }
}

```

```
return 0;
}
```

**Lab Work III (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)**

- Given the under-listed initialized original array elements (values), code in C++ to output the values in your output. NB: (1) The array size was not given. (2) The elements are to be inputted as specified. (3) The output is to be in the reverse order of the inputs  
my values = {8, 5, 3, 5, 7, 2, 3, 4, 5, 6, 1, 17, 11, 19}
- Given  
my box = {4, 3, 8, 5, 7, 2}, sort and output this array elements in DESCENDING ORDER (i.e. from highest to lowest)

**Coding Sheet (where this is not sufficient, please kindly print out and append)**

Program ID/No: \_\_\_\_\_ Start Time: \_\_\_\_\_ Run Time: \_\_\_\_\_

| Program Statement: |  |
|--------------------|--|
| 1.                 |  |
| 2.                 |  |
| 3.                 |  |
| 4.                 |  |
| 5.                 |  |
| 6.                 |  |
| 7.                 |  |
| 8.                 |  |
| 9.                 |  |
| 10.                |  |
| 11.                |  |
| 12.                |  |
| 13.                |  |
| 14.                |  |
| 15.                |  |
| 16.                |  |
| 17.                |  |
| 18.                |  |
| 19.                |  |
| 20.                |  |
| 21.                |  |
| 22.                |  |
| 23.                |  |
| 24.                |  |
| 25.                |  |

**Matrix**

Matrix is declared thus:

`int mymatrix[3][4]`, meaning a matrix containing 3 rows of elements and 4 columns of elements

e.g.

|   |   |   |   |
|---|---|---|---|
| 6 | 7 | 9 | 5 |
| 5 | 4 | 3 | 9 |
| 1 | 4 | 5 | 8 |

**Initialization**

If initialized, the format is:

`mymatrix = {2,5,6,8,4},{9,0,3,2,1},{8,9,3,4,7}`

Which implies a matrix of 3 rows and 5 columns

e.g.

|   |   |   |   |   |
|---|---|---|---|---|
| 2 | 5 | 6 | 8 | 4 |
| 9 | 0 | 3 | 2 | 1 |
| 8 | 9 | 3 | 4 | 7 |

**Programming Illustration XII****Inputting and Outputting Matrix (e.g. Row 3, Column 4)**

```
#include <iostream>
using namespace std;
int main(int argc, char** argv) {
    int mymatrix[3][4];
    //inputting....
    for(int i=0; i<3; i++)
    {
        for(int j=0; j<4; j++)
        {
            cout<<"Input Element in Row "<<i+1<<" Column "<<j+1<<" : "; cin>>mymatrix[i][j];
        }
    }
    //outputting...

    for(int i=0; i<3; i++)
    {
        cout<<endl;
        for(int j=0; j<4; j++)
        {
            cout<<mymatrix[i][j]<<" ";
        }
    }
    return 0;
}
```

**Programming Illustration XIII****Transposing a Matrix**

```
#include <iostream>
using namespace std;
/* run this program using the console pauser or add your own getch, system("pause") or input loop */
int main(int argc, char** argv) {
    int m, n;
    cout<<"Specify the matrix row dimension : "; cin>>m;
    cout<<"Specify the matrix column dimension : "; cin>>n;

    int ourmatrix[m][n];
```

```

//inputting...
for(int i=0;i<m;i++)
{
    for(int j=0;j<n;j++)
    {
        cout<<"Input Element in Row "<<i+1<<" Column "<<j+1<<" : ";cin>>ourmatrix[i][j];
    }
}
//outputting...

cout<<"\n"<<"This is the original matrix: "<<endl;
for(int i=0;i<m;i++)
{
    cout<<endl;
    for(int j=0;j<n;j++)
    {
        cout<<ourmatrix[i][j]<<" ";
    }
}

cout<<"\n"<<"And lo, the transpose of the original matrix: "<<endl;
for(int i=0;i<m;i++)
{
    cout<<endl;
    for(int j=0;j<n;j++)
    {
        cout<<ourmatrix[j][i]<<" ";
    }
}
return 0;
}

```

#### Program Illustration XIV

##### Arithmetic Operations on Matrix

```

#include <iostream>
using namespace std;
/* run this program using the console pauser or add your own getch, system("pause") or input loop */
int main(int argc, char** argv) {
    int m,n;
    cout<<"Specify the matrix row dimension: "; cin>>m;
    cout<<"Specify the matrix column dimension "; cin>>n;

    int hismatrix[m][n],hermatrix[m][n],theirmatrix[m][n];

    //inputting....
    for(int i=0;i<m;i++)
    {
        for(int j=0;j<n;j++)
        {
            cout<<"Input Element in Row "<<i+1<<" Column "<<j+1<<" for Hismatrix: ";cin>>hismatrix[i][j];
            cout<<"Input Element in Row "<<i+1<<" Column "<<j+1<<" for Hermatrix: ";cin>>hermatrix[i][j];
            theirmatrix[i][j]=hismatrix[i][j] + hermatrix[i][j];
        }
    }
    //outputting....
}

```

```

cout<<"\n"<<"HISMATRIX: "<<endl;
for(inti=0;i<m;i++)
{
    cout<<endl;
    for(intj=0;j<n;j++)
    {
        cout<<hismatrix[i][j]<<" ";
    }
}
cout<<"\n"<<"HERMATRIX: "<<endl;
for(inti=0;i<m;i++)
{
    cout<<endl;
    for(intj=0;j<n;j++)
    {
        cout<<hermatrix[i][j]<<" ";
    }
}
cout<<"\n"<<"MARRYING THEM TOGETHER, THEIRMATRIX: "<<endl;
for(inti=0;i<m;i++)
{
    cout<<endl;
    for(intj=0;j<n;j++)
    {
        cout<<theirmatrix[i][j]<<" ";
    }
}
return 0;
}

```

**Lab Work IV** (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)

1. Modify the matrix arithmetic operation by:
  - i. Code the inputting of hismatrix and hermatrix elements separately
  - ii. Performs subtraction operation on the two matrices to give theirdiffmatrix
  - iii. Perform division operation on the two matrices to give theirdivmatrix
  - iv. Compute and output the inverse of each matrix

2. Given

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 0 \end{bmatrix}$$

and

$$B = [2, 3]$$

Compute and output:

- i.  $A * B$       ii.  $A/B$       iii.  $A^2$

3. Capture the below given value for Matrix A:

$$A = \begin{bmatrix} 2 & 3 & 4 \\ 5 & 6 & 7 \\ 8 & 9 & 0 \end{bmatrix}$$

Then, compute and output:

i. row addition of Matrix A's elements

e.g. 9  
18  
17

ii. Column addition of Matrix A's elements

*Coding Sheet (where this is not sufficient, please kindly print out and append)*

Program ID/No: \_\_\_\_\_ Start Time: \_\_\_\_\_ Run Time: \_\_\_\_\_

| Program Statement: |  |
|--------------------|--|
| 1.                 |  |
| 2.                 |  |
| 3.                 |  |
| 4.                 |  |
| 5.                 |  |
| 6.                 |  |
| 7.                 |  |
| 8.                 |  |
| 9.                 |  |
| 10.                |  |
| 11.                |  |
| 12.                |  |
| 13.                |  |
| 14.                |  |
| 15.                |  |
| 16.                |  |
| 17.                |  |
| 18.                |  |
| 19.                |  |
| 20.                |  |
| 21.                |  |
| 22.                |  |
| 23.                |  |
| 24.                |  |
| 25.                |  |

## **Paper 6 of 6 — Cyssa hub200 lvi materialsCOS 201holiday\_drill\_01022026.pptx**

Lecture/Notes · Google Drive harvest

---